



TUTO2 : DÉVELOPPEMENT DE VISUALISATIONS DE DONNÉES AVEC D3.JS

NICOLAS MÉDOC LUXEMBOURG INSTITUTE OF SCIENCE AND TECHNOLOGY

Plan



1. Transformation d'échelle : de l'espace des données à l'espace visuel

2. Interactions

Les fonctions de transformation d'échelle (d3-scale)



- Les échelles permettent d'appliquer des transformations entre l'espace des données (valeurs du domaine) et un intervalle de valeurs correspondant à un encodage visuel choisi
- Différents types d'échelles sont disponibles selon le type de transformation : linéaire, logarithmique, ordinale, quantile, etc.
- <https://d3js.org/d3-scale>

Exercice : utiliser des échelles appropriées pour associer les données à des variables visuelles



*Dans le jeu de données généré, chaque cellule représente une entreprise qui vend un certain nombre de produits (`nbProductSold`) dans différents pays. Un second attribut est **salesGrowth**, correspondant à un taux de croissance (positif ou négatif) des ventes.*

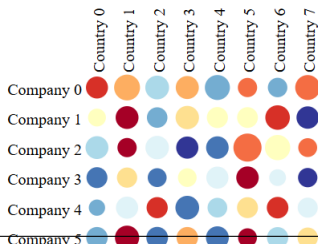
<https://github.com/nicolasmedoc/Tuto2-D3js>

Exercice : utiliser des échelles appropriées pour associer les données à des variables visuelles



Dans le jeu de données généré, chaque cellule représente une entreprise qui vend un certain nombre de produits (`nbProductSold`) dans différents pays. Un second attribut est **salesGrowth**, correspondant à un taux de croissance (positif ou négatif) des ventes.

<https://github.com/nicolasmedoc/Tuto2-D3js>





Taille et couleur des cellules

```
// build the size scale
const radiusMin = 2;
const radiusMax = cellSize / 2;
const minNbProductSold = d3.min(genData.map(cellData=>cellData.nbProductSold));
const maxNbProductSold = d3.max(genData.map(cellData=>cellData.nbProductSold));
const cellSizeScale = d3.scaleLinear()
    .domain([minNbProductSold, maxNbProductSold])
    .range([radiusMin, radiusMax-1])
;
// build the color scale
const colorScheme = d3.schemeRdYlBu[11];
const cellColorScale = d3.scaleQuantile()
    .domain(genData.map(cellData=>cellData.salesGrowth))
    .range(colorScheme)
;
```

Exemple : utilisation des fonctions d'échelle pour encoder la taille et la couleur des cercles



Pour calculer les attributs des cercles (rayon et couleur), on déclare une fonction prenant l'élément de données en paramètre :

```
...  
    .attr("r",(cellData)=>cellSizeScale(cellData.nbProductSold))  
    .attr("fill",(cellData) =>{  
        const color = cellColorScale(cellData.salesGrowth);  
        return color;  
    })  
...
```



Ajout d'interactions

En D3.js, vous pouvez déclarer des événements avec la fonction `.on()` :

```
.on("click",(event, cellData)=>{// do something with event and/or cellData})  
.on("mouseenter",(event, cellData)=>{// do something with event/cellData})  
.on("mouseleave",(event, cellData)=>{// do something with event/cellData})
```


Exercice 2 : afficher le contour d'une cellule sur l'événement clic



```
function renderMatrix(genData)
...
  .append("g")
  .attr("class", "cellG")
  .attr("transform", (cellData)=>{
    return "translate("+(cellData.colPos*cellSize)+ ...
  })
  .on("click", (event, cellData)=>{
    handleOnClickCell(cellData);
  })
;
...
cellG.append("circle")
  .attr("class", "CellCircle")
  .attr("stroke", "black")
  .attr("stroke-width", (cellData)=>cellData.selected?2:0)
```

Exercice 2 : afficher le contour d'une cellule sur l'événement clic



```
function handleClickCell(cellData){
  genData=genData.map(item=>{
    if (item.index===cellData.index){
      return {...item,selected:!cellData.selected};
    }else{
      return item;
    }
  })
  renderMatrix(genData);
}
```

Exercice 2 : afficher le contour d'une cellule sur l'événement clic



Puisque `genData` est mis à jour mais qu'aucun nouvel élément n'est ajouté, la sélection `enter()` est vide et rien ne se produit lors de l'appel à `renderMatrix()` suite à l'événement de clic.

Une solution rapide mais peu optimale consiste à supprimer tous les éléments et à reconstruire la visualisation avec un nouveau data binding :

```
...  
function removeMatrix(){  
    matSvgG.selectAll('*').remove();  
}  
function renderMatrix(genData) {  
    removeMatrix();  
    ...
```

Cependant, ce n'est pas optimal. Il est préférable de ne modifier que

~~l'élément associé à la donnée mise à jour. Cela est possible en utilisant le~~



General update pattern avec `join()`

Le "general update pattern" de D3.js permet de déclarer différents comportements après la liaison des données. La fonction **`join()`**, appelée juste après la fonction `data()` de liaison de données, prend en paramètre trois fonctions :

- une **fonction enter** pour définir le comportement des nouveaux éléments ;
- une **fonction update** pour gérer les éléments correspondant à la liaison précédente ;
- une **fonction exit** pour les anciens éléments qui n'existent plus.

Voir les illustrations : <https://bost.ocks.org/mike/selection/#enter-update-exit> et <https://observablehq.com/@d3/selection-join>.

Exercice 3 : implementation du "general update pattern"



Dans `renderMatrix`, éviter d'appeler `removeMatrix()` et appeler la fonction `join()` en suivant l'exemple ci-dessous :

```
const cellG = this.matSvg.selectAll(".cellG")
  .data(genData,(cellData)=>cellData.index)
  .join(
    enter =>{// appends elements with fixed attributes
      // append cellG
      // append CellRect with the color
      // append CellCircle at center position
    },
    update =>{ // select elements and declare changing attributes
      // the cell position (<g> translation)
      // the circle size
      // the circle color
    },
    exit =>{ // declare what to do with items that don't exist anymore
      exit.remove();
    }
  )
```



Exercice 4 : optimisation des mises à jour

Dans certains cas, on souhaite ne mettre à jour qu'un nombre limité d'éléments, par exemple pour mettre en évidence une ou plusieurs cellules cliquées ou survolées. Dans ce cas, il n'est pas nécessaire de modifier l'affichage de toutes les cellules. On crée donc une fonction spécifique pour déclarer ce comportement à l'aide l'approche du "update pattern" :

```
function handleClickCell(cellData){
    const cellsToUpdate=[{...cellData,selected:!cellData.selected}]
    updateCellHighlighting(cellsToUpdate)
}
...
function updateCellHighlighting(cellsToUpdate){
    matSvgG.selectAll(".cellG")
        .data(cellsToUpdate, cellData=>cellData.index)
    // no need to call join() because we don't need enter or exit
    // update selection is already returned by data()
    .select(".CellCircle")
    .attr("stroke-width",cellData=>cellData.selected?2:0);
}
```

Exercice 5 : utilisation de transitions animées



Avant de supprimer des éléments ou de mettre à jour des attributs, on peut déclarer une transition animée avec une durée spécifique afin d'observer de manière fluide les transitions lors des changements de position ou de couleur :

```
cellG.transition()  
  .duration(transitionDuration)  
  .attr("transform",(cellData)=>{  
    return "translate("+ (cellData.colPos*this.cellSize)...  
  })  
;
```

Ajouter des transitions avant les phases de mise à jour et de sortie du join.



Exercice 5 (optionnel)

Utiliser les événements `mouseenter` et `mouseleave` pour implémenter l'interaction de survol de la souris sur les cellules de la matrice.

```
.on("mouseenter",(event, cellData)=>{// do something with event/cellData})  
.on("mouseleave",(event, cellData)=>{// do something with event/cellData})
```




Exercice 6 (optionnel)

Dans `renderMatrix()`, ajouter des labels dans les marges supérieure et gauche.



Exercice 7 (optionnel)

- Lorsque l'utilisateur clique sur les étiquettes des pays, trier les produits par ordre décroissant de nbProduced
- Lorsque l'utilisateur clique sur les étiquettes des entreprises, trier les pays par ordre décroissant de nbProduced